

Adventist University of Central Africa

Master's in Big Data Analytics

Chapter 5 — Resampling Methods Lab

Introduction to Statistical Learning with Applications in R

Programme: MSc Big Data Analytics
Course Title: Data Mining
Course Code: MSDA 9124
Lecturer: Dr. Pacifique Nizeyimana
Student Name: Wayne Rubangisa
Student ID: 101220

September 2026

1 Introduction

When a model is fitted, its training error is usually too optimistic: the same observations were used both to estimate the model and to measure how well it fits. The real questions are different. How well will the model predict new observations? How much would its coefficients change if we had collected a different sample? Which model complexity gives the best balance between underfitting and overfitting?

Resampling methods answer these questions by repeatedly reusing the observed data in carefully chosen ways. This lab develops four closely related ideas:

1. the validation-set approach;
2. leave-one-out cross-validation (LOOCV);
3. k -fold cross-validation; and
4. the bootstrap.

The first three estimate prediction error. The bootstrap estimates the sampling variability of statistics such as regression coefficients and portfolio weights. We use the `Auto` and `Portfolio` data sets from ISLR2. The main prediction problem is to model automobile fuel efficiency (`mpg`) from horsepower using polynomial regression.

```
library(ISLR2)
library(boot)
```

1.1 The prediction problem and the bias–variance trade-off

Let Y be a response and X a predictor. A fitted model produces $\hat{f}(X)$, and its prediction error on a new observation can be written as

$$\mathbb{E} \left[(Y - \hat{f}(X))^2 \mid X = x \right] = \underbrace{\sigma^2}_{\text{irreducible noise}} + \underbrace{\text{Bias} \left[\hat{f}(x) \right]^2}_{\text{systematic error}} + \underbrace{\text{Var} \left[\hat{f}(x) \right]}_{\text{sensitivity to the sample}}.$$

A very simple model may have high bias because it cannot represent the true relationship. A highly flexible model may have low training error but high variance because it follows random details of the particular sample. The purpose of resampling is to estimate the out-of-sample part of this trade-off using only the data we have.

2 Validation Set Approach

2.1 Idea and mathematical definition

The validation-set approach randomly divides the data into a training set and a validation set. We fit the model using only the training observations, then evaluate predictions on observations that were not used for fitting. For regression, the estimate is the validation mean squared error (MSE):

$$\text{MSE}_{\text{val}} = \frac{1}{n_{\text{val}}} \sum_{i \in \mathcal{V}} \left(y_i - \hat{f}_{\mathcal{T}}(x_i) \right)^2,$$

where $\mathcal{T}$ is the training set and $\mathcal{V}$ is the validation set. The notation on $\hat{f}_{\mathcal{T}}$ matters: the model was fitted using the training observations only.

The approach is easy and fast, but the result depends on one random split. Also, because the model is trained on fewer observations than are ultimately available, its validation error can be a little pessimistic.

2.2 One training/validation split

Auto contains 392 cars after missing observations have been removed. We compare polynomial models of degrees one, two, and three. A fixed seed makes the random split reproducible for the report.

```
data(Auto)
Auto <- na.omit(Auto)
n <- nrow(Auto)

set.seed(1)
train <- sample(seq_len(n), size = n / 2)
validation <- -train

validation_mse <- function(model, data, validation_indices) {
  mean((data$mpg - predict(model, newdata = data))[validation_indices]^2)
}

validation_results <- data.frame(
  Degree = 1:3,
  Validation_MSE = c(
    validation_mse(lm(mpg ~ horsepower, data = Auto, subset = train), Auto, validation),
    validation_mse(lm(mpg ~ poly(horsepower, 2), data = Auto, subset = train), Auto, validation),
    validation_mse(lm(mpg ~ poly(horsepower, 3), data = Auto, subset = train), Auto, validation)
  )
)
validation_results
```

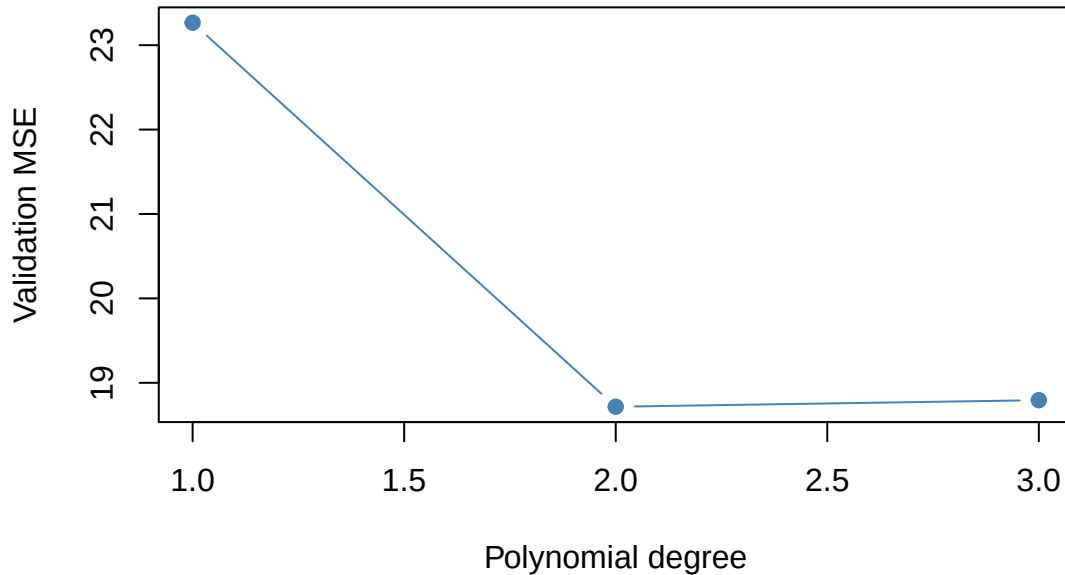
```
##   Degree Validation_MSE
## 1      1      23.26601
## 2      2      18.71646
## 3      3      18.79401
```

The model with the smallest validation MSE is preferred for this split. The quadratic model normally improves substantially over the linear model because fuel efficiency falls quickly at first and then levels off. If the cubic model is not better, that is evidence that its extra flexibility is fitting sample-specific variation rather than a reliable pattern.

```
plot(
  validation_results$Degree,
  validation_results$Validation_MSE,
  type = "b", pch = 19, col = "steelblue",
  xlab = "Polynomial degree", ylab = "Validation MSE",
```

```
main = "Validation error for polynomial models"
)
```

Validation error for polynomial models



2.3 Why one split is not enough

The validation set is itself a random sample. Changing the split changes both the fitted coefficients and the observations used to judge them. The model comparison is therefore noisy even when the underlying data-generating process has not changed.

```
validation_mse_for_seed <- function(seed, degree) {
  set.seed(seed)
  training_indices <- sample(seq_len(n), size = n / 2)
  model_formula <- if (degree == 1) {
    mpg ~ horsepower
  } else {
    mpg ~ poly(horsepower, degree)
  }
  model <- lm(model_formula, data = Auto, subset = training_indices)
  mean(
    (Auto$mpg - predict(model, newdata = Auto))[-training_indices]^2
  )
}

split_results <- expand.grid(Seed = 1:10, Degree = 1:3)
split_results$Validation_MSE <- mapply(
  validation_mse_for_seed,
  split_results$Seed,
  split_results$Degree
)
```

```
aggregate(Validation_MSE ~ Degree, data = split_results,
          FUN = function(x) c(Mean = mean(x), SD = sd(x)))
```

```
##   Degree Validation_MSE.Mean Validation_MSE.SD
## 1      1           24.101279           2.096181
## 2      2           19.097242           1.543873
## 3      3           19.245682           1.619390
```

The standard deviation across splits shows the weakness of relying on one holdout set. Repeated splitting is informative, but cross-validation gives a more systematic way to ensure that every observation participates in both training and validation.

3 Leave-One-Out Cross-Validation

3.1 Procedure

LOOCV creates n training sets. For the i th fit, observation i is left out, the model is fitted to the other $n - 1$ observations, and the held-out prediction error is recorded. The final estimate is

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{f}^{(-i)}(x_i) \right)^2,$$

where $\hat{f}^{(-i)}$ means that observation i was excluded when fitting. Unlike a single validation split, every observation is used for validation exactly once and for training $n - 1$ times.

For linear regression there is a useful shortcut. If $e_i = y_i - \hat{y}_i$ is the ordinary residual and h_{ii} is the leverage of observation i , then

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(\frac{e_i}{1 - h_{ii}} \right)^2.$$

The `cv.glm()` function uses this idea for a generalized linear model. An ordinary Gaussian `glm()` is equivalent to `lm()` for this purpose.

```
loocv_results <- data.frame(
  Degree = 1:10,
  LOOCV_MSE = NA_real_
)

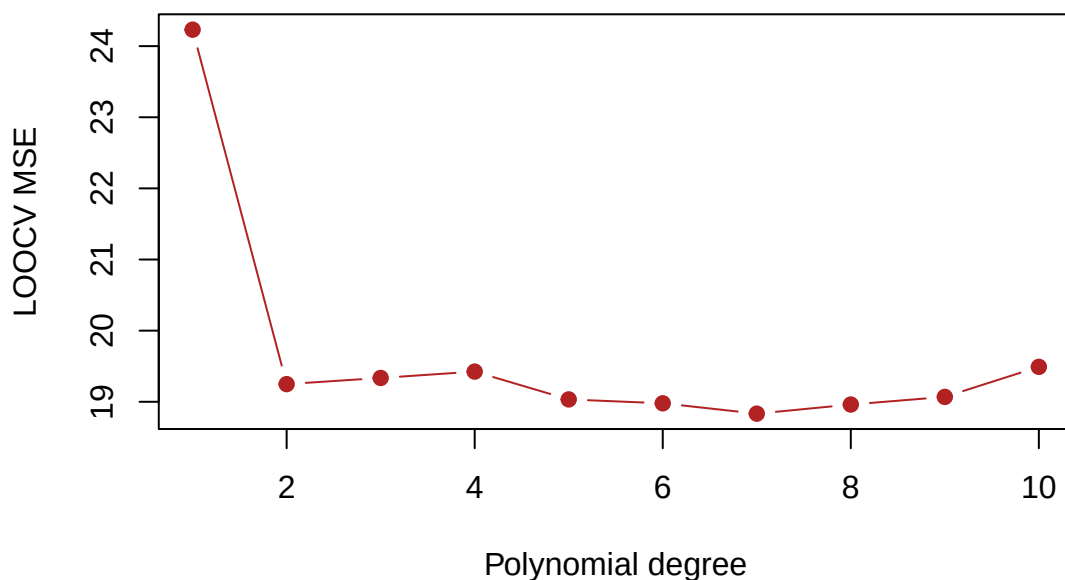
for (degree in loocv_results$Degree) {
  model_formula <- if (degree == 1) {
    mpg ~ horsepower
  } else {
    mpg ~ poly(horsepower, degree)
  }
  model <- glm(model_formula, data = Auto)
  loocv_results$LOOCV_MSE[degree] <- cv.glm(Auto, model)$delta[1]
}
```

```
loocv_results
```

```
##      Degree LOOCV_MSE
## 1         1  24.23151
## 2         2  19.24821
## 3         3  19.33498
## 4         4  19.42443
## 5         5  19.03321
## 6         6  18.97864
## 7         7  18.83305
## 8         8  18.96115
## 9         9  19.06863
## 10        10  19.49093
```

```
plot(
  loocv_results$Degree, loocv_results$LOOCV_MSE,
  type = "b", pch = 19, col = "firebrick",
  xlab = "Polynomial degree", ylab = "LOOCV MSE",
  main = "Leave-one-out cross-validation"
)
```

Leave-one-out cross-validation



The typical pattern is a large improvement from degree one to degree two, followed by a flat curve. That means the quadratic term captures important curvature, while higher-order terms do not produce a dependable reduction in prediction error. LOOCV is attractive because each fit uses almost the full data set, but it requires many model fits and can be computationally costly for large data sets.

4 k -Fold Cross-Validation

4.1 Procedure and trade-off

In k -fold cross-validation, the observations are divided into k folds. Each fold is held out once while the other $k - 1$ folds are used for training:

$$CV_{(k)} = \frac{1}{k} \sum_{j=1}^k \left(\frac{1}{|\mathcal{V}_j|} \sum_{i \in \mathcal{V}_j} \left(y_i - \hat{f}^{(-j)}(x_i) \right)^2 \right).$$

When $k = n$, this becomes LOOCV. Common choices such as $k = 5$ or $k = 10$ require far fewer fits while still using every observation for validation. Smaller k can have slightly more bias because each training set is smaller, but it is usually more computationally efficient.

```
set.seed(17)
kfold_results <- data.frame(
  Degree = 1:10,
  Tenfold_CV_MSE = NA_real_
)

for (degree in kfold_results$Degree) {
  model_formula <- if (degree == 1) {
    mpg ~ horsepower
  } else {
    mpg ~ poly(horsepower, degree)
  }
  model <- glm(model_formula, data = Auto)
  kfold_results$Tenfold_CV_MSE[degree] <-
    cv.glm(Auto, model, K = 10)$delta[1]
}

kfold_results

##      Degree Tenfold_CV_MSE
## 1         1      24.27207
## 2         2      19.26909
## 3         3      19.34805
## 4         4      19.29496
## 5         5      19.03198
## 6         6      18.89781
## 7         7      19.12061
## 8         8      19.14666
## 9         9      18.87013
## 10        10      20.95520

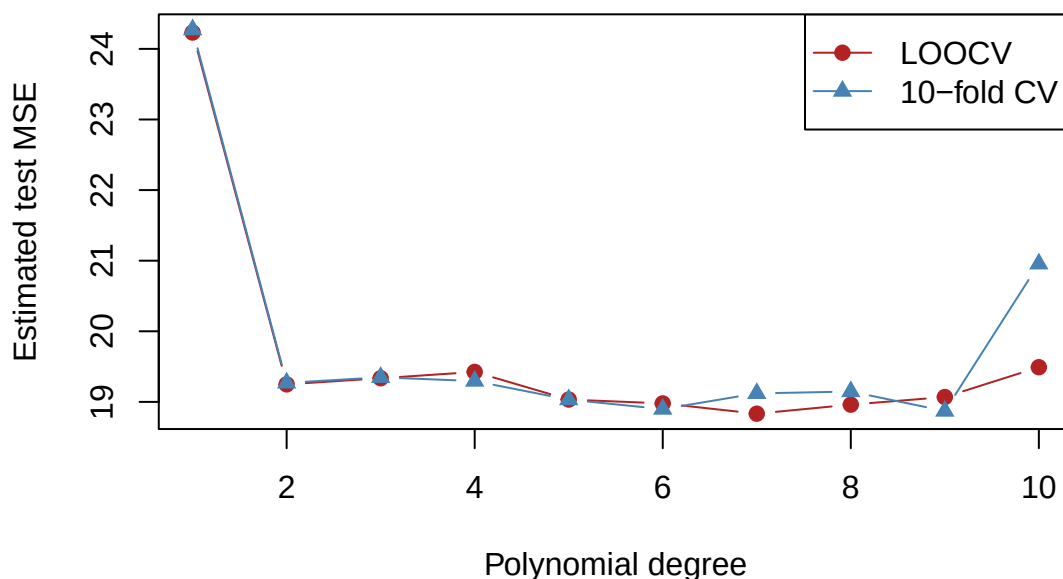
plot(
  loocv_results$Degree, loocv_results$LOOCV_MSE,
  type = "b", pch = 19, col = "firebrick",
  xlab = "Polynomial degree", ylab = "Estimated test MSE",
```

```

ylim = range(c(loocv_results$LOOCV_MSE, kfold_results$Tenfold_CV_MSE)),
main = "LOOCV and 10-fold CV"
)
lines(
  kfold_results$Degree, kfold_results$Tenfold_CV_MSE,
  type = "b", pch = 17, col = "steelblue"
)
legend(
  "topright", legend = c("LOOCV", "10-fold CV"),
  col = c("firebrick", "steelblue"), pch = c(19, 17), lty = 1
)

```

LOOCV and 10-fold CV



The two curves should have the same practical conclusion even though their individual values differ slightly. Ten-fold CV is often the useful default: it is much cheaper than LOOCV and generally gives a stable estimate of the model's test error. Cross-validation estimates prediction error; it does not prove that one model is the true data-generating model.

5 The Bootstrap

5.1 The basic idea

Cross-validation focuses on prediction. The bootstrap focuses on uncertainty in an estimate. We treat the observed sample as an approximation to the population and repeatedly draw bootstrap samples of size n **with replacement**. For a statistic $\hat{\theta} = T(Z_1, \dots, Z_n)$, the b th bootstrap estimate is

$$\hat{\theta}^{*b} = T(Z_1^{*b}, \dots, Z_n^{*b}), \quad b = 1, \dots, B.$$

The standard deviation of the bootstrap estimates approximates the standard error:

$$\widehat{\text{SE}}_{\text{boot}}(\hat{\theta}) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{\theta}^{*b} - \bar{\theta}^*)^2}.$$

The phrase *with replacement* is essential. It means some observations may appear multiple times and some may be absent in a particular bootstrap sample. That variation imitates the fact that a new random sample would not contain exactly the same observations as the original one.

5.2 Portfolio allocation

The `Portfolio` data set contains returns for two assets, `X` and `Y`. Suppose we invest fraction α in asset `X` and $1 - \alpha$ in asset `Y`. The portfolio return is

$$R_p = \alpha X + (1 - \alpha)Y.$$

Minimizing its variance gives

$$\hat{\alpha} = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}.$$

This formula is easy to calculate on the original data. The uncertainty in $\hat{\alpha}$ is less obvious, so it is a good bootstrap example.

```
data(Portfolio)

alpha_function <- function(data, index) {
  X <- data$X[index]
  Y <- data$Y[index]
  (var(Y) - cov(X, Y)) /
  (var(X) + var(Y) - 2 * cov(X, Y))
}

alpha_hat <- alpha_function(Portfolio, seq_len(nrow(Portfolio)))

set.seed(7)
alpha_boot <- boot(Portfolio, alpha_function, R = 1000)
alpha_se <- sd(alpha_boot$t)
alpha_ci <- alpha_hat + c(-1, 1) * 1.96 * alpha_se

data.frame(
  Estimate = alpha_hat,
  Bootstrap_SE = alpha_se,
  CI_lower = alpha_ci[1],
  CI_upper = alpha_ci[2]
)
```

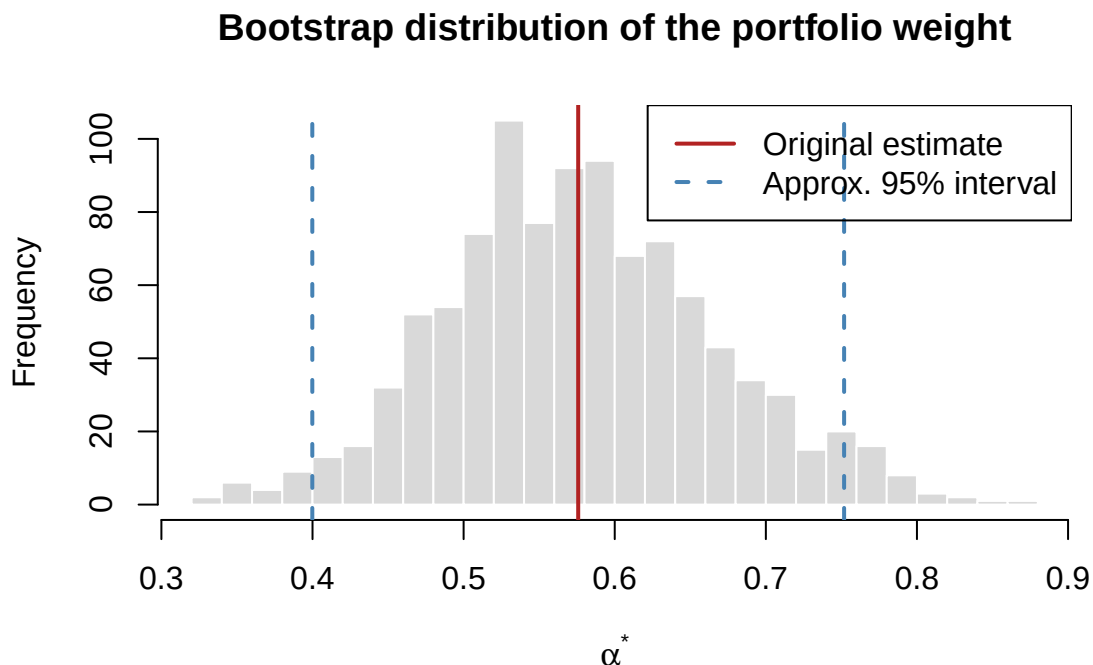
```
##      Estimate Bootstrap_SE CI_lower CI_upper
## 1 0.5758321    0.08978204 0.3998593 0.7518049
```

The estimate is the suggested fraction invested in asset X , conditional on the observed sample. The bootstrap standard error says how much this allocation would typically move across comparable samples. The approximate normal-theory interval is computed as

$$\hat{\alpha} \pm 1.96 \widehat{\text{SE}}_{\text{boot}}.$$

This interval is an uncertainty statement, not a guarantee that future returns will lie inside it. It describes uncertainty in the estimated allocation, while the investment decision itself also requires a risk model and practical constraints.

```
hist(
  alpha_boot$t, breaks = 30, col = "grey85", border = "white",
  xlab = expression(alpha^"*"),
  main = "Bootstrap distribution of the portfolio weight"
)
abline(v = alpha_hat, col = "firebrick", lwd = 2)
abline(v = alpha_ci, col = "steelblue", lty = 2, lwd = 2)
legend(
  "topright",
  legend = c("Original estimate", "Approx. 95% interval"),
  col = c("firebrick", "steelblue"), lty = c(1, 2), lwd = 2
)
```



5.3 Bootstrap standard errors for regression coefficients

The bootstrap can estimate uncertainty for almost any statistic that can be computed from a data set. Here the statistic is the vector of coefficients from a regression of `mpg` on `horsepower`.

```

coefficient_function <- function(data, index) {
  coef(lm(mpg ~ horsepower, data = data, subset = index))
}

coefficient_hat <- coefficient_function(Auto, seq_len(nrow(Auto)))

set.seed(1)
coefficient_boot <- boot(Auto, coefficient_function, R = 1000)

coefficient_summary <- data.frame(
  Term = names(coefficient_hat),
  Estimate = unname(coefficient_hat),
  Bootstrap_SE = apply(coefficient_boot$t, 2, sd),
  Classical_SE = coef(summary(lm(mpg ~ horsepower, data = Auto)))[, "Std. Error"]
)
coefficient_summary

```

```

##              Term      Estimate Bootstrap_SE Classical_SE
## (Intercept) (Intercept) 39.9358610  0.843931305  0.717498656
## horsepower  horsepower -0.1578447  0.007367396  0.006445501

```

The slope is negative: cars with greater horsepower tend to have lower fuel efficiency. The bootstrap standard error measures the variability of that slope under repeated samples. Comparing it with the usual regression standard error is a useful diagnostic. Similar values suggest that the classical approximation is reasonable here; noticeable differences remind us that standard errors depend on assumptions about the model and its errors.

We can also bootstrap a nonlinear specification. The quadratic coefficient measures curvature, so its uncertainty helps us decide whether the observed curvature is stable enough to support the added model complexity.

```

quadratic_function <- function(data, index) {
  coef(lm(mpg ~ horsepower + I(horsepower^2),
    data = data, subset = index))
}

set.seed(1)
quadratic_boot <- boot(Auto, quadratic_function, R = 1000)

data.frame(
  Term = names(coef(lm(mpg ~ horsepower + I(horsepower^2), data = Auto))),
  Estimate = coef(lm(mpg ~ horsepower + I(horsepower^2), data = Auto)),
  Bootstrap_SE = apply(quadratic_boot$t, 2, sd)
)

```

```

##              Term      Estimate Bootstrap_SE
## (Intercept)      (Intercept) 56.900099702 2.0300222526
## horsepower      horsepower -0.466189630 0.0324241984
## I(horsepower^2) I(horsepower^2) 0.001230536 0.0001172164

```

6 Putting the Methods Together

The methods answer different questions, so their outputs should not be confused:

Method	Main question	Mechanism
Validation set	How well does this model predict on one held-out split?	One train/validation split
LOOCV	What is the average held-out error when each observation is left out?	n fits
k -fold CV	What is held-out error using a practical number of folds?	k fits
Bootstrap	How variable is an estimate across plausible samples?	Resample with replacement

For the **Auto** analysis, the resampling results generally support a quadratic relationship between **mpg** and **horsepower**: degree two improves on degree one, while higher degrees provide little dependable improvement. This is a model-selection conclusion about predictive performance, not a claim that a quadratic law is physically exact.

The broader lesson is that a fitted model is only one estimate obtained from one sample. Validation and cross-validation expose how prediction changes when observations are held out. The bootstrap exposes how statistical summaries change when the observed sample is perturbed. Together they provide an empirical way to reason about generalization and uncertainty rather than reporting training fit alone.

7 Conclusion

Resampling methods turn a single data set into a collection of informative experiments. The validation set is simple but unstable. LOOCV uses data efficiently but can be expensive. Ten-fold cross-validation is often a good practical compromise for comparing model complexity. The bootstrap is more general: it estimates standard errors for statistics whose sampling distributions are difficult to derive analytically.

The most important habit is to match the resampling method to the question. Use held-out predictions when the concern is generalization, and use bootstrap replicates when the concern is the uncertainty of an estimate. Neither method removes irreducible noise, but both make the limits of what we learn from a finite sample much more visible.